

Exercise 1: Setting Up JUnit

Steps

Step 1: Create a New Java Project

1. Open your IDE (IntelliJ IDEA or Eclipse).
2. Select **File** → **New Project**.
3. Choose **Java Project**.
4. Enter the project name (e.g., `JUnitDemo`).
5. Click **Finish**.

Step 2: Add JUnit Dependency

If you are using **Maven**, add the following dependency inside the `<dependencies>` section of your `pom.xml` file.

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

Step 3: Create a Java Class to Test

Create a class named `Calculator.java`.

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }
}
```

Step 4: Create a Test Class

Create a new class named `CalculatorTest.java` inside the `test` package.

```
import static org.junit.Assert.assertEquals;

import org.junit.Test;

public class CalculatorTest {

    @Test
    public void testAddition() {

        Calculator calculator = new Calculator();

        int result = calculator.add(10, 20);

        assertEquals(30, result);
    }
}
```

Expected Output

Tests run: 1
Failures: 0
Errors: 0

BUILD SUCCESSFUL

or in IntelliJ/Eclipse:

✓ testAddition() PASSED

Exercise 3: Assertions in JUnit

Steps

1. Create a test class.
2. Import the JUnit assertion methods.
3. Write test cases using different assertions such as `assertEquals()`, `assertTrue()`, `assertFalse()`, `assertNull()`, and `assertNotNull()`.

Solution Code

```
import static org.junit.Assert.*;

import org.junit.Test;

public class AssertionsTest {

    @Test

    public void testAssertions() {

        // Assert Equals

        assertEquals(5, 2 + 3);

        // Assert True

        assertTrue(5 > 3);

        // Assert False

        assertFalse(5 < 3);

        // Assert Null

        assertNull(null);

        // Assert Not Null

        assertNotNull(new Object());

    }

}
```

Expected Output

Tests run: 1

Failures: 0

Errors: 0

BUILD SUCCESSFUL

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Steps

1. Create the class to be tested.
2. Create a test class.
3. Use `@Before` to initialize test objects.
4. Write test methods following the Arrange-Act-Assert (AAA) pattern.
5. Use `@After` to clean up resources after each test.

Class Under Test

```
public class Calculator {  
  
    public int add(int a, int b) {  
  
        return a + b;  
  
    }  
  
    public int multiply(int a, int b) {  
  
        return a * b;  
  
    }  
  
}
```

Test Class

```
import static org.junit.Assert.*;  
  
import org.junit.After;  
  
import org.junit.Before;
```

```
import org.junit.Test;

public class CalculatorTest {

    private Calculator calculator;

    @Before

    public void setUp() {

        System.out.println("Setting up test...");

        calculator = new Calculator();

    }

    @After

    public void tearDown() {

        System.out.println("Cleaning up after test...");

        calculator = null;

    }

    @Test

    public void testAddition() {

        // Arrange

        int a = 10;

        int b = 20;

        // Act

        int result = calculator.add(a, b);

        // Assert

        assertEquals(30, result);

    }

}
```

```
@Test
public void testMultiplication() {
    // Arrange
    int a = 5;
    int b = 6;
    // Act
    int result = calculator.multiply(a, b);
    // Assert
    assertEquals(30, result);
}
}
```

Expected Output

Setting up test...

Cleaning up after test...

Setting up test...

Cleaning up after test...

Tests run: 2

Failures: 0

Errors: 0

BUILD SUCCESSFUL